

Table of contents

Task 1: The Long-Running Process.....	3
Task 2: Process Identity (PID and PPID).....	4
Task 3: Exit Codes and OS Feedback.....	5
Task 4: Standard I/O Streams.....	6
Task 5: Conditional Execution and Termination.....	7

List of figures

Figure 1: C Program for Long-Running Process.....	4
Figure 2: Run In Background.....	4
Figure 3: Monitor The Process.....	4
Figure 4: C Program for Process Identity (PID and PPID).....	5
Figure 5: Terminal Commands.....	5
Figure 6: C Program for Exit Codes.....	6
Figure 7: Checking Exit Status.....	6
Figure 8: C Program for Standard Input and Output.....	7
Figure 9: Running Task 4.....	7
Figure 10: C Program for Conditional Execution and Termination.....	8
Figure 11: Testing Control Flow.....	8

Task 1: The Long-Running Process

```
#include <stdio.h>
#include <unistd.h>

int main() {
    printf("I am starting....");

    for(int i = 1; i <= 30 ; i++){
        sleep(1);
    }
    printf("I am finished.\n");
    return 0;
}
```

Figure 1: C Program for Long-Running Process

```
shriya-awale@shriya-awale-QEMU-Virtual-Machine:~/batch38/lab3$ gcc task1_alive.c -o task1
shriya-awale@shriya-awale-QEMU-Virtual-Machine:~/batch38/lab3$ ./task1
I am starting....I am finished.
shriya-awale@shriya-awale-QEMU-Virtual-Machine:~/batch38/lab3$ ./task1 &
[2] 3294
```

Figure 2: Run In Background

```
shriya-awale@shriya-awale-QEMU-Virtual-Machine:~/batch38/lab3$ ps aux | grep task1
shriya-+  3304  0.0  0.0 10308 2128 pts/0    S+   21:34   0:00 grep --color=auto task1
```

Figure 3: Monitor The Process

Summary

The program created a process that remained active for 30 seconds. Its execution was monitored using the `ps` command to observe the running process.

Task 2: Process Identity (PID and PPID)

```
#include <stdio.h>
#include <unistd.h>

int main() {

    pid_t my_pid = getpid();

    pid_t my_ppid = getppid();

    printf("My PID is %d\n", my_pid);
    printf("My Parent PID is %d\n", my_ppid);

    printf("Sleeping for 20 seconds.....\n");
    sleep(20);

    return 0;
}
```

Figure 4: C Program for Process Identity (PID and PPID)

```
shriya-awale@shriya-awale-QEMU-Virtual-Machine:~/batch38/lab3$ touch task2_identity.c
shriya-awale@shriya-awale-QEMU-Virtual-Machine:~/batch38/lab3$ nano task2_identity.c
shriya-awale@shriya-awale-QEMU-Virtual-Machine:~/batch38/lab3$ gcc task2_identity.c -o task2

shriya-awale@shriya-awale-QEMU-Virtual-Machine:~/batch38/lab3$ ./task2 &
[1] 3465
My PID is 3465
My Parent PID is 2685
Sleeping for 20 seconds.....
shriya-awale@shriya-awale-QEMU-Virtual-Machine:~/batch38/lab3$ ps -p 3375 -o pid,ppid,cmd
  PID   PPID  CMD
shriya-awale@shriya-awale-QEMU-Virtual-Machine:~/batch38/lab3$ ps -p 3465 -o pid,ppid,cmd
  PID   PPID  CMD
 3465   2685 ./task2
```

Figure 5: Terminal Commands

Summary

The program displayed its PID and PPID to show its process identity. The process information was then verified using the ps command.

Task 3: Exit Codes and OS Feedback

```
#include <stdio.h>

int main() {
    int num;
    printf("Enter a number (positive for success, negative for fail): ");
    scanf("%d", &num);
    if (num > 0) {
        printf("Success\n");
        return 0;
    } else {
        printf("Failure\n");
        return 1;
    }
}
```

Figure 6: C Program for Exit Codes

```
shriya-awale@shriya-awale-QEMU-Virtual-Machine:~/batch38/lab3$ nano task3_exit.c
shriya-awale@shriya-awale-QEMU-Virtual-Machine:~/batch38/lab3$ gcc task3_exit.c -o task3
shriya-awale@shriya-awale-QEMU-Virtual-Machine:~/batch38/lab3$ ./task3
Enter a number (positive for success, negative for fail): 5
Success
shriya-awale@shriya-awale-QEMU-Virtual-Machine:~/batch38/lab3$ echo $?
0
shriya-awale@shriya-awale-QEMU-Virtual-Machine:~/batch38/lab3$ ./task3
Enter a number (positive for success, negative for fail): -5
Failure
shriya-awale@shriya-awale-QEMU-Virtual-Machine:~/batch38/lab3$ echo $?
1
```

Figure 7: Checking Exit Status

Summary

The program accepted a number and returned different exit codes based on the result. Exit code 0 indicated success, while 1 indicated failure.

Task 4: Standard I/O Streams

```
#include <stdio.h>

int main () {
    char name [50];

    printf("Enter the name: ");
    scanf("%s", name);

    printf("Hello, %s! Welcome to OS class.\n",
    name);
    return 0;
}
```

Figure 8: C Program for Standard Input and Output

```
shriya-awale@shriya-awale-QEMU-Virtual-Machine:~/batch38/lab3$ nano task4_input.c
shriya-awale@shriya-awale-QEMU-Virtual-Machine:~/batch38/lab3$ gcc task4_input.c -o task4
shriya-awale@shriya-awale-QEMU-Virtual-Machine:~/batch38/lab3$ ./task4
Enter the name: Shriya
Hello, Shriya! Welcome to OS class.
```

Figure 9: Running Task 4

Summary

The program asked the user for their name and then displayed a greeting. It showed how basic input and output work in C.

Task 5: Conditional Execution and Termination

```
#include <stdio.h>
#include <unistd.h>

int main() {
    int choice;

    printf("Current PID: %d\n", getpid());
    printf("Do you want to continue? (1 for Yes, 0 for No): ");

    scanf("%d", &choice);

    if(choice == 1){
        printf("Continuing.....\n");
        sleep(5);
        return 0;
    } else {
        printf("Exiting.....\n");
        return 1;
    }
}
```

Figure 10: C Program for Conditional Execution and Termination

```
shriya-awale@shriya-awale-QEMU-Virtual-Machine:~/batch38/lab3$ touch task5_control.c
shriya-awale@shriya-awale-QEMU-Virtual-Machine:~/batch38/lab3$ nano task5_control.c
shriya-awale@shriya-awale-QEMU-Virtual-Machine:~/batch38/lab3$ gcc task5_control.c -o task5
shriya-awale@shriya-awale-QEMU-Virtual-Machine:~/batch38/lab3$ ./task5
Current PID: 4671
Do you want to continue? (1 for Yes, 0 for No): 1
Continuing.....
shriya-awale@shriya-awale-QEMU-Virtual-Machine:~/batch38/lab3$ echo $?
0
shriya-awale@shriya-awale-QEMU-Virtual-Machine:~/batch38/lab3$ ./task5
Current PID: 4688
Do you want to continue? (1 for Yes, 0 for No): 0
Exiting.....
shriya-awale@shriya-awale-QEMU-Virtual-Machine:~/batch38/lab3$ echo $?
1
```

Figure 11: Testing Control Flow

Summary

The program used user input to decide whether to continue or terminate. It also displayed the process ID and returned an appropriate exit status.